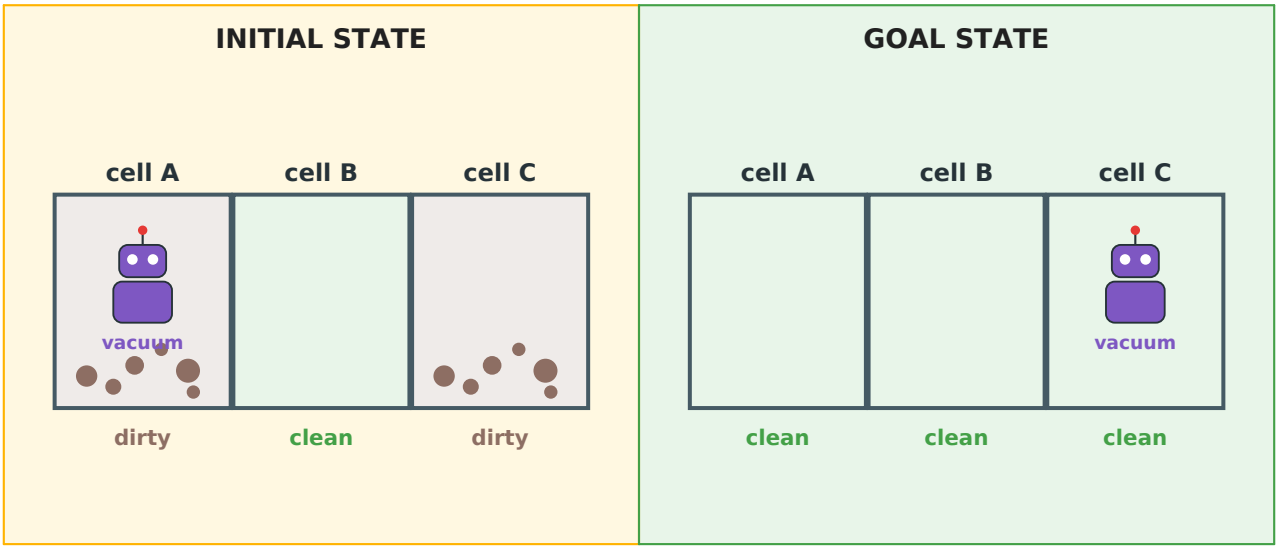
Problem 2: Vacuum Cleaner World

Level: **Very basic** · Folder: **02_Vacuum_Cleaner/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

This is the classic agent example from Russell & Norvig. A vacuum-cleaning robot works on a corridor of three cells **A - B - C**. Cells A and C are dirty and B is already clean. The robot starts in A. It can move to a neighbouring cell or **suck** up the dirt in its current cell. The goal is that **every cell is clean**.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- A domain with **two different actions**.
- Goals that are a **conjunction**: `(and ...)`.
- Using a pair of opposite predicates (*dirty* / *clean*) instead of negation.
- A goal fact can already be true at the start (clean B).

2. Formulating the problem in PDDL

Step 1: Types and objects. One type `cell`; three objects A, B, C.

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(vacuum-at ?c)`	The vacuum robot is on cell ?c.
`(adjacent ?c1 ?c2)`	Cells ?c1 and ?c2 are neighbours (static).
`(dirty ?c)`	Cell ?c contains dirt.
`(clean ?c)`	Cell ?c is clean.

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect
move `?from - cell, ?to - cell` Go to a neighbouring cell.	`(vacuum-at ?from)` `(adjacent ?from ?to)`	`+ (vacuum-at ?to)` `− (vacuum-at ?from)`
suck `?c - cell` Remove dirt from the current cell. Requires the cell to be dirty. It becomes clean.	`(vacuum-at ?c)` `(dirty ?c)`	`+ (clean ?c)` `− (dirty ?c)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

INITIAL STATE (8 facts)	GOAL

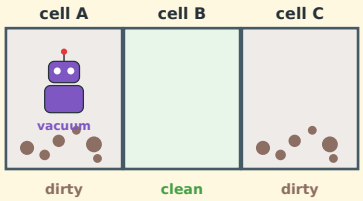
`(vacuum-at A)`
`(adjacent A B)`
`(adjacent B A)`
`(adjacent B C)`
`(adjacent C B)`
`(dirty A)`
`(clean B)`
`(dirty C)`
	`(clean A)`
	`(clean B)`
	`(clean C)`

Modelling notes

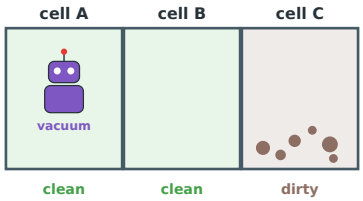
- Plain STRIPS does not allow `(not (dirty ?c))` inside a precondition or goal. So we keep a second predicate **clean**, and the effect of **suck** keeps the pair consistent (add clean, delete dirty).
- Goals describe the **final state**, not jobs to do. `(clean B)` is already true, so the planner does nothing in B. It only passes through.
- The planner sucks A first because the robot is already there. Going right first would cost one extra move.

3. Solution plan (4 actions, optimal)

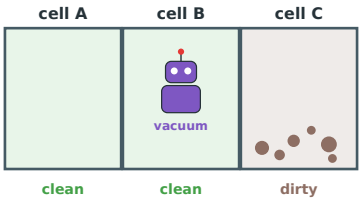
The optimal plan has 4 actions: suck, move, move, suck. The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



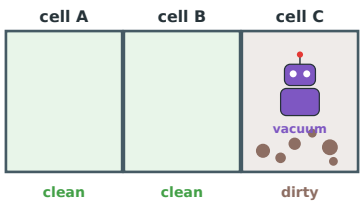
Initial state



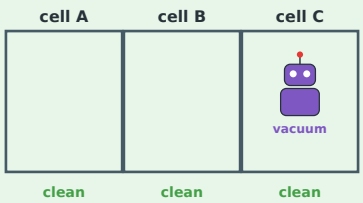
Step 1
`(suck a)`



Step 2
`(move a b)`



Step 3
`(move b c)`



Step 4
`(suck c)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. Make cell B dirty as well. What is the new optimal plan length?
2. Extend the world to a 2 x 2 grid (four cells) and write the new **adjacent** facts.
3. Add a **battery**: a predicate `(charged)` required by suck and deleted by it, plus a **recharge** action that is possible only in cell A.
4. Start the vacuum in cell B. Which side does the planner clean first?

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 2 : Vacuum Cleaner World
;; Level     : Very basic
;; A vacuum robot moves along a row of cells and sucks up dirt.
;; ============================================================
(define (domain vacuum-world)
  (:requirements :strips :typing)

  (:types cell)

  (:predicates
    (vacuum-at ?c - cell)          ; the vacuum is on cell ?c
    (adjacent ?c1 - cell ?c2 - cell) ; ?c1 and ?c2 are neighbours
    (dirty ?c - cell)              ; cell ?c has dirt
    (clean ?c - cell)              ; cell ?c is clean
  )

  ;; Move to a neighbouring cell
  (:action move
    :parameters (?from - cell ?to - cell)
    :precondition (and (vacuum-at ?from)
                       (adjacent ?from ?to))
    :effect (and (vacuum-at ?to)
                 (not (vacuum-at ?from)))
  )

  ;; Suck the dirt in the current cell
  (:action suck
    :parameters (?c - cell)
    :precondition (and (vacuum-at ?c)
                       (dirty ?c))
    :effect (and (clean ?c)
                 (not (dirty ?c)))
  )
)
```

problem.pddl

```
;; Three cells in a row:  [ A ] - [ B ] - [ C ]
;; Vacuum starts in A. Cells A and C are dirty, B is clean.
(define (problem vacuum-1)
  (:domain vacuum-world)

  (:objects A B C - cell)

  (:init
    (vacuum-at A)
    (adjacent A B) (adjacent B A)
    (adjacent B C) (adjacent C B)
    (dirty A)
    (clean B)
    (dirty C)
  )

  (:goal (and (clean A)
              (clean B)
              (clean C)))
)
```

solution_plan.txt

```
1. (suck a)
2. (move a b)
3. (move b c)
4. (suck c)
```